

Phase 6 — 测试体系

SPEC-014 | Status: 设计中 | 依赖: SPEC-002, SPEC-003, SPEC-004, SPEC-006, SPEC-005, SPEC-008, SPEC-010, SPEC-011, SPEC-012, SPEC-013, SPEC-009, SPEC-007

目标

前置依赖检查

前置 Spec	状态	备注
SPEC-002	✓/✗	CI Pipeline 就绪
SPEC-003	✓/✗	基础设施可用
SPEC-004	✓/✗	Agent 核心可用 (含安全审计)
SPEC-006	✓/✗	知识库系统可用
SPEC-005	✓/✗	Artifact 存储可用
SPEC-007	✓/✗	数据分析 Logic 可用
SPEC-008	✓/✗	全部 Skill 可用
SPEC-010	✓/✗	统计监控可用
SPEC-011	✓/✗	IM 集成可用
SPEC-012	✓/✗	Hermes 可用
SPEC-013	✓/✗	管理后台可用
SPEC-009	✓/✗	任务队列/调度器可用

建立完整测试体系：单元测试 >70% 覆盖率、集成测试、E2E 测试、性能优化、安全测试。

背景

Roadmap Phase 6 (Week 11-12), P6-01 ~ P6-09, 总计 ~50h。不包含生产部署 (MVP 阶段不需要)。

详细设计

1. 单元测试

- `go test ./internal/... -v -count=1 -cover`
- 目标覆盖率 > 70%
- 重点: Repository 层、Logic 层、Skill 执行逻辑

2. 集成测试

- Service 层 + 数据库交互 (Docker Compose 环境)
- `go test ./tests/integration/... -v -tags=integration`

3. E2E 测试

- Playwright 完整流程: 登录 → Chat 查询 → Agent 任务创建 → 知识库搜索 → 管理后台
- 用例编号 UI-001 ~ UI-020 (逐步替换占位 UI-000)
- `data-testid` 属性全覆盖

4. 代码质量

- golangci-lint 全量检查 + 修复
- SonarQube 扫描 (sonar-check CI 步骤)
- 依赖安全扫描 (govulncheck)

5. 性能优化

- 压测工具 (wrk/vegeta)
- 瓶颈识别: 数据库慢查询、大对象分配、goroutine 泄漏
- 优化目标: Chat API P95 < 3s, Agent 异步任务队列堆积 < 100

6. 安全测试

- OWASP Top 10 渗透测试
- JWT 令牌安全验证
- RBAC 权限绕过检测

- SQL 注入防护验证

可行性分析

检查项	结论
是否需要新 DB 集合	No
是否影响现有 API	No
性能影响	优化后性能提升
是否需要新增 Skill	No
是否需要 E2E 测试	Yes（需要编写全部 E2E 用例）

相关文件

文件	角色	变更幅度
tests/ui/*.spec.ts	E2E 用例	新建（逐步替换占位）
tests/integration/	集成测试	新建

UI Test / E2E 验收规则

此 spec 是 E2E 用例集中的编写阶段。

- [] UI-001 ~ UI-010: Chat 模式全流程
- [] UI-011 ~ UI-015: Agent 任务管理流程
- [] UI-016 ~ UI-020: 管理后台页面
- [] 全部 E2E 用例在 sonar-check + ui-tests 均通过

验证标准

1. 单元测试覆盖率 > 70%
2. 集成测试覆盖核心 Service 层流程

3. E2E 全部用例通过 (≥ 20 个)
4. SonarQube Quality Gate 通过
5. 无高危安全漏洞